Date:15-10-2026

CO1-ASSESSMENT TOOL 3-CODING CHALLENGE

COURSE CODE: DSA0308

COURSE NAME: NATURAL LANGUAGE PROCESSING

REGISTRATION NO:192472193 **NAME:** C G CHARISHMA

Question 1: Intelligent Email and Password Validator Using Regex

Aim:

To develop a Python program using regular expressions to validate email addresses, passwords, and mobile numbers.

Algorithm:

- Start the program.
- Read the email address, password, and mobile number from the user.
- Use regular expressions to check whether the email starts with an alphabet and has a valid domain and extension.
- Check whether the password contains at least eight characters, an uppercase letter, a lowercase letter, a digit, and a special character.
- Check whether the mobile number contains exactly 10 digits and starts with a digit from 6 to 9.
- Display the validation results.
- Stop the program

Program Code:

```
import re


def validate_email(email):
    pattern = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$"
    return re.fullmatch(pattern, email) is not None
```

```
def validate_password(password):
    pattern = r"(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}"
    return re.fullmatch(pattern, password) is not None

def validate_mobile(mobile):
    pattern = r"^[6-9]\d{9}$"
    return re.fullmatch(pattern, mobile) is not None

email = input("Enter email address: ")
password = input("Enter password: ")
mobile = input("Enter mobile number: ")

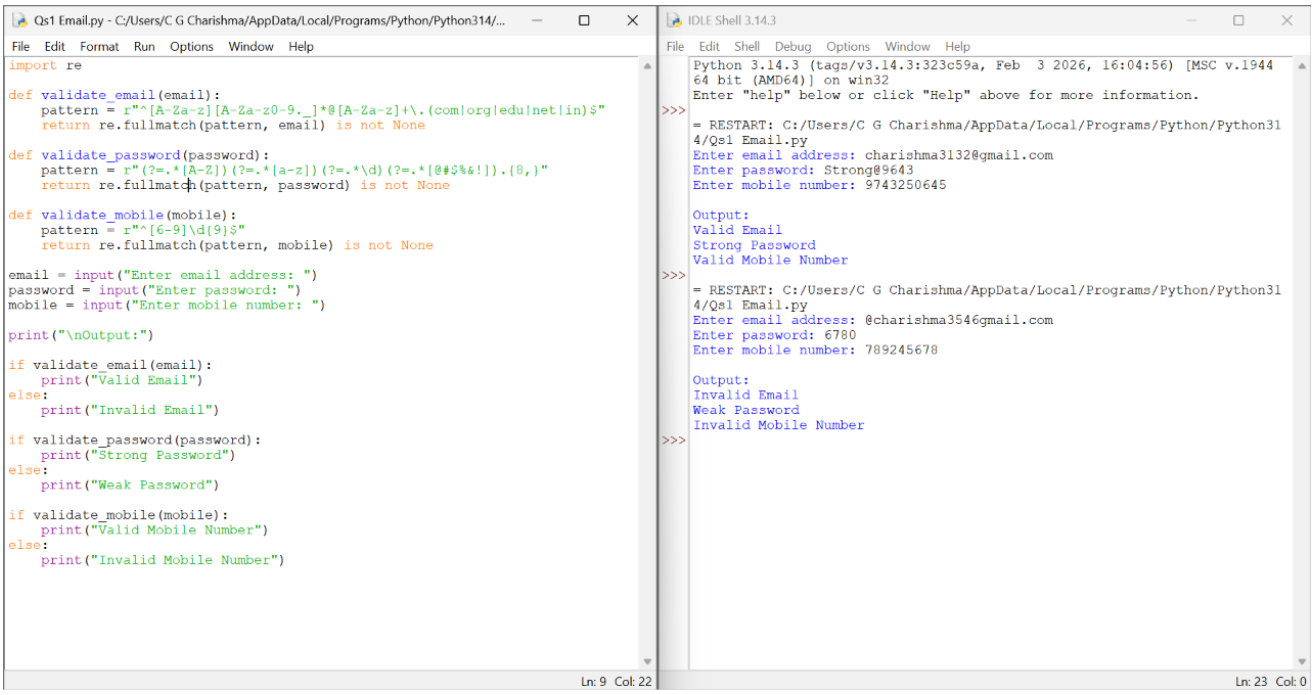
print("\nOutput:")

if validate_email(email):
    print("Valid Email")
else:
    print("Invalid Email")

if validate_password(password):
    print("Strong Password")
else:
    print("Weak Password")

if validate_mobile(mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

Output:



Result:

Thus, the Python program was implemented successfully to validate email addresses, passwords, and mobile numbers using regular expressions.

Question 2: DFA Simulator

Aim:

To develop a Python program to simulate a Deterministic Finite Automaton (DFA) and determine whether an input string is accepted or rejected.

Algorithm:

- Start the program.
- Define the states, input alphabet, transition table, initial state, and final states.
- Validate the DFA configuration.
- Read the input string from the user.
- Begin from the initial state and process each input symbol using the transition table.
- Display the transition path followed by the DFA.
- Check whether the final state reached belongs to the set of final states.
- Display Accepted if the final state is a final state; otherwise, display Rejected.
- Stop the program.

Program Code:

```
class DFA:
    def __init__(self, states, alphabet, transitions,
                 initial_state, final_states):
        self.states = set(states)
        self.alphabet = set(alphabet)
        self.transitions = transitions
        self.initial_state = initial_state
        self.final_states = set(final_states)
```

```
        self.validate()

    def validate(self):
        if not self.states:
            raise ValueError("States cannot be empty")

        if not self.alphabet:
            raise ValueError("Alphabet cannot be empty")

        if self.initial_state not in self.states:
            raise ValueError("Invalid initial state")

        if not self.final_states.issubset(self.states):
            raise ValueError("Invalid final state")

        for state in self.states:
            for symbol in self.alphabet:
                key = (state, symbol)

                if key not in self.transitions:
                    raise ValueError("Missing transition")

                if self.transitions[key] not in self.states:
                    raise ValueError("Invalid destination state")

    def simulate(self, text):
```

```
        state = self.initial_state
        path = [state]

        for symbol in text:
            if symbol not in self.alphabet:
                print("Transition Path:", " -> ".join(path))
                print("Rejected - Invalid Symbol:", symbol)
                return

            state = self.transitions[(state, symbol)]
            path.append(state)

        print("Transition Path:", " -> ".join(path))

        if state in self.final_states:
            print("Accepted")
        else:
            print("Rejected")


def read_values(message):
    return [
        value.strip()
        for value in input(message).split(",")
        if value.strip()
    ]
```

```
def main():
    print("1. Example DFA")
    print("2. Custom DFA")

    choice = input("Choose option: ")

    try:
        if choice == "1":
            states = ["q0", "q1", "q2"]
            alphabet = ["a", "b"]

            transitions = {
                ("q0", "a"): "q1",
                ("q0", "b"): "q0",
                ("q1", "a"): "q1",
                ("q1", "b"): "q2",
                ("q2", "a"): "q1",
                ("q2", "b"): "q0"
            }

            dfa = DFA(
                states, alphabet, transitions,
                "q0", ["q2"]
            )
```

```
elif choice == "2":
    states = read_values("Enter states: ")
    alphabet = read_values("Enter alphabet: ")

    initial = input("Enter initial state: ")
    final_states = read_values("Enter final states: ")

    transitions = {}

    for state in states:
        for symbol in alphabet:
            destination = input(
                f"Transition {state} with {symbol}: "
            )
            transitions[(state, symbol)] = destination.strip()

    dfa = DFA(
        states, alphabet, transitions,
        initial, final_states
    )

else:
    print("Invalid choice")
    return
```

```
        count = int(input("Enter number of strings: "))

        if count < 1:
            print("Enter a positive number")
            return

        for i in range(count):
            text = input("Enter string: ")
            dfa.simulate(text)
            print()

    except ValueError as error:
        print("Error:", error)

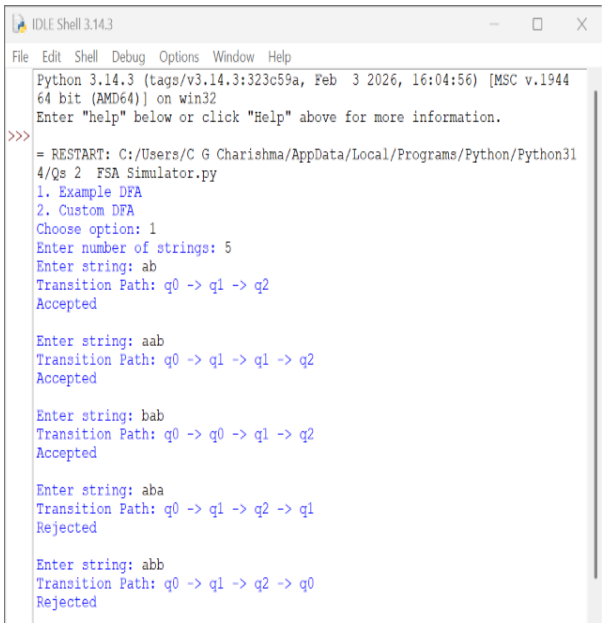

if __name__ == "__main__":
    main()
```

Output:










```
>>>
========== RESTART: C:/Users/C G Charishma/AppData/Local/Programs/Python/Python314/Qs 2  FSA Simulator.py ==========
1. Example DFA
2. Custom DFA
Choose option: 2
Enter states: q0,q1,q2
Enter alphabet: a,b
Enter initial state: q0
Enter final states: q2
Transition q0 with a: q1
Transition q0 with b: q0
Transition q1 with a: q1
Transition q1 with b: q2
Transition q2 with a: q1
Transition q2 with b: q0
Enter number of strings: 3
Enter string: ab
Transition Path: q0 -> q1 -> q2
Accepted

Enter string: aab
Transition Path: q0 -> q1 -> q1 -> q2
Accepted

Enter string: aba
Transition Path: q0 -> q1 -> q2 -> q1
Rejected
```

Result:

Thus, the Python program was implemented successfully to simulate a DFA, display its transition path, and determine whether input strings are accepted or rejected.

Question 3: Regex-Based Text Search Engine

Aim:

To develop a Python program using regular expressions to search text for dates, phone numbers, hashtags, mentions, prefixes, suffixes, and exact words.

Algorithm:

- Start the program.
- Read multiple lines of text from the user until END is entered.
- Display the available search options.
- Read the user's search choice.
- Apply the corresponding regular expression to find matching patterns in the text.
- For prefix, suffix, and exact-word searches, read the required search term.
- Display all matching results or a message if no matches are found.
- Allow the user to perform another search or exit the program.
- Stop the program.

Program Code:

```
import re


def search_text(text, choice, query=""):
    patterns = {
        "1": r"(?<!\d)(?:\d{1,2}/\d{1,2}/\d{4}|\d{4}-\d{2}-\d{2})(?!\d)",
        "2": r"(?<!\d)[6-9]\d{9}(?!\d)",
        "3": r"(?<!\w)#[A-Za-z0-9_]+",
        "4": r"(?<!\w)@[A-Za-z0-9_]+"
    }


    if choice in patterns:
```

```
        return re.findall(patterns[choice], text)

    if choice not in ["5", "6", "7"]:
        return None

    if not query:
        return []

    query = re.escape(query)

    if choice == "5":
        pattern = r"\b" + query + r"[A-Za-z0-9_]*"
    elif choice == "6":
        pattern = r"\b[A-Za-z0-9_]*" + query + r"\b"
    else:
        pattern = r"\b" + query + r"\b"

    return re.findall(pattern, text, re.IGNORECASE)


def main():
    print("Enter text. Type END on a separate line to finish.")

    lines = []

    while True:
```

```
    line = input()

    if line == "END":
        break

    lines.append(line)

text = "\n".join(lines)

print("\n1. Search Date")
print("2. Search Phone Number")
print("3. Search Hashtag")
print("4. Search Mention")
print("5. Search Prefix")
print("6. Search Suffix")
print("7. Search Exact Word")

choice = input("Select option: ")
query = ""

if choice in ["5", "6", "7"]:
    query = input("Enter word, prefix, or suffix: ")

matches = search_text(text, choice, query)

if matches is None:
```

```
        print("Invalid choice")

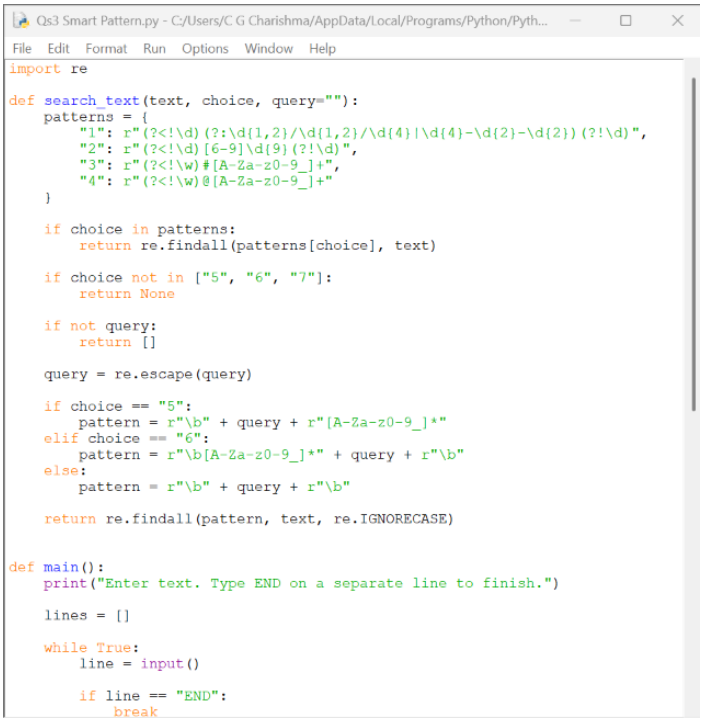
    elif matches:
        print("Matching Results:")

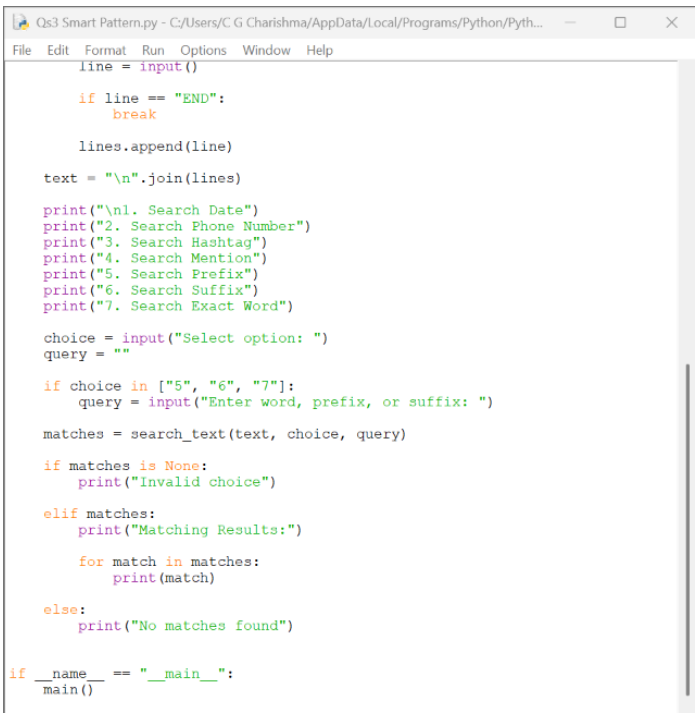
        for match in matches:
            print(match)

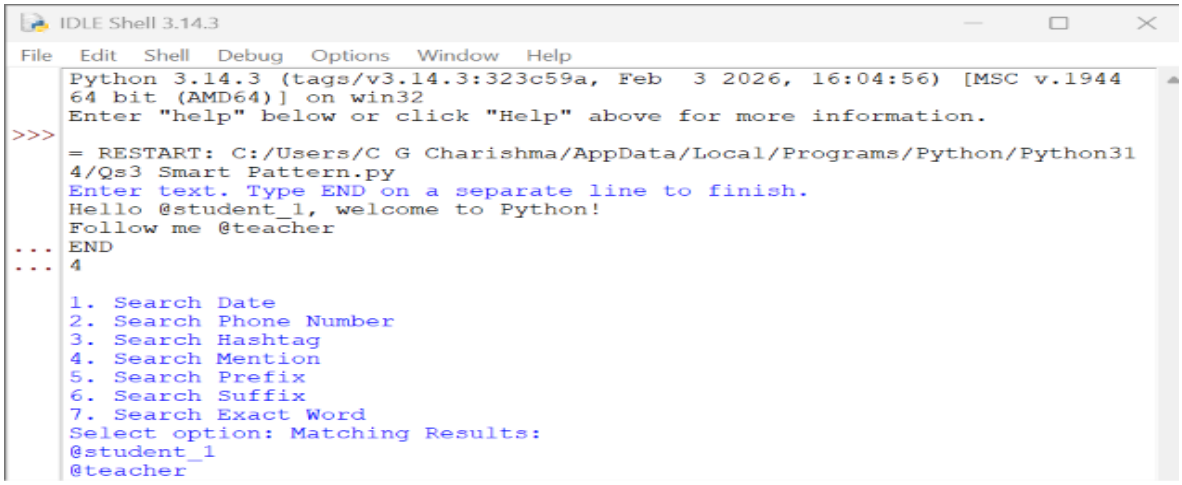
    else:
        print("No matches found")


if __name__ == "__main__":
    main()
```

Output:







Result:

Thus, the Python program was implemented successfully to search text for different patterns using regular expressions and display the matching results.